



Shortest Path Algorithms

Abdullah All Mamun Siam

Lecturer, Dhaka International University(DIU)

Graph

1. A graph is a collection of nodes (or vertices) connected by edges.
2. Graphs can be directed (edges have directions) or undirected (edges have no direction).
3. Edges may also have weights, representing the cost, distance, or time required to traverse them.

Applications:

1. Navigation Systems (Google Maps, GPS)
2. Network Routing
3. Robotics and AI Path Planning

Shortest Path Algorithm

A Shortest Path Algorithm is a computational method used to find the path between two nodes in a graph such that the sum of the weights of the edges in the path is minimized.

Types of Problems Solved:

1.Single Source Shortest Path :

Find the shortest path from one source node to all other nodes in the graph.

Example: Dijkstra's Algorithm, Bellman-Ford Algorithm.

2.All-Pairs Shortest Path :

Find the shortest paths between every pair of nodes in the graph.

Example: Floyd-Warshall Algorithm.

Single Source Shortest Path

The Single Source Shortest Path problem is a computational problem where the goal is to determine the shortest path from a given source node to all other nodes in a graph.

1.Dijkstra's Algorithm

2.Bellman-Ford Algorithm.

Dijkstra Algorithm

Dijkstra Algorithm is a greedy algorithm for solving the Single Source Shortest Path problem on a graph with non-negative edge weights. It finds the shortest path from a given source node to all other nodes in the graph.

Dijkstra Algorithm

Steps of Dijkstra's Algorithm:

1. Mark the source node with a current distance of 0 ($d[s]=0$) and assign infinity (∞) to all other nodes.
2. Set the unvisited node with the smallest current distance as the current node (u).
3. For each neighbor v of u :

$$\begin{aligned} &\text{If } (d[u] + c[u, v] < d[v]) \\ &\quad d[v] = d[u] + c[u, v] \end{aligned}$$

Here:

$c[u, v]$: Cost (weight) of the edge from u to v .

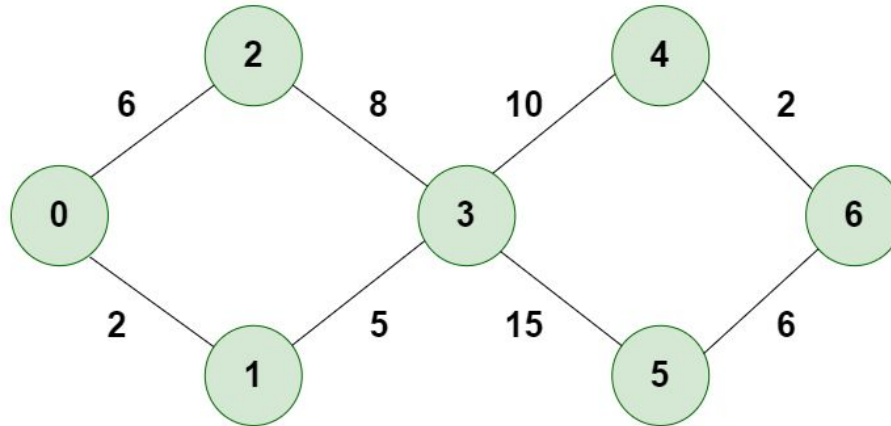
$d[v]$: Distance to v .

$d[u]$: Distance to u

4. Mark the current node u as visited.
5. Go back to Step 2 if there are any unvisited nodes.

Example of Dijkstra Algorithm

Example Graph

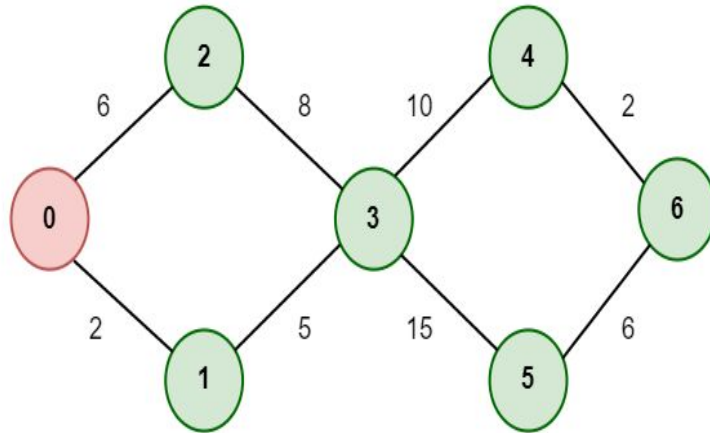


Dijkstra's Algorithm

Example of Dijkstra Algorithm

STEP 1

Start from Node 0 and mark Node 0 as Visited and check for adjacent nodes



Unvisited Nodes

{0,1,2,3,4,5,6}

Distance:

0: 0 ✓

1: ∞

2: ∞

3: ∞

4: ∞

5: ∞

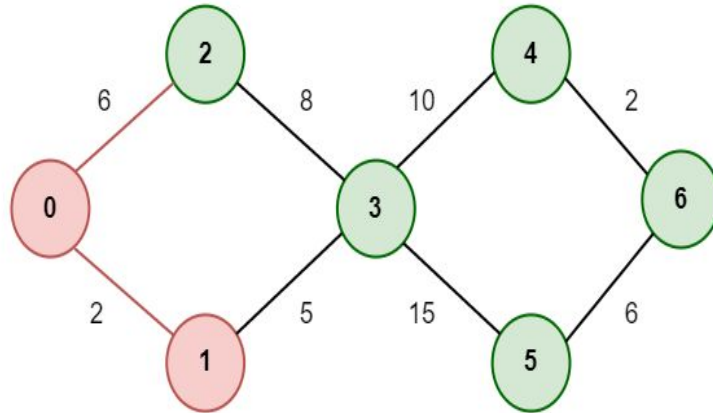
6: ∞

Dijkstra's Algorithm

Example of Dijkstra Algorithm

STEP 2

Mark Node 1 as Visited and add the Distance



Unvisited Nodes

{0,1,2,3,4,5,6}

Distance:

0: 0 ✓

1: 2 ✓

2: ∞

3: ∞

4: ∞

5: ∞

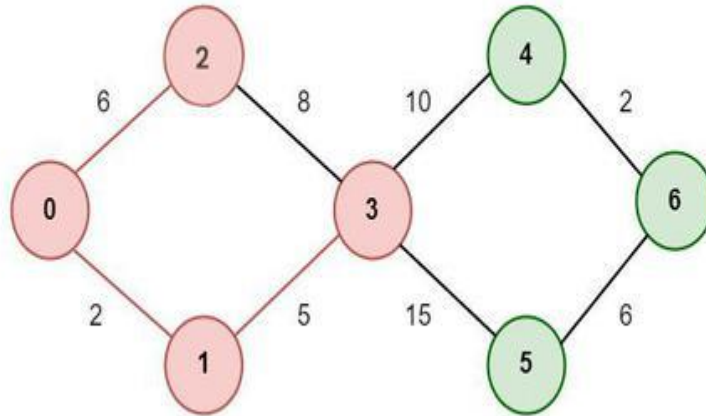
6: ∞

Dijkstra's Algorithm

Example of Dijkstra Algorithm

STEP 3

Mark Node 3 as Visited after considering the Optimal path and add the Distance



Unvisited Nodes

{0,1,2,3,4,5,6}

Distance:

0: 0 ✓

1: 2 ✓

2: 6 ✓

3: 7 ✓

4: ∞

5: ∞

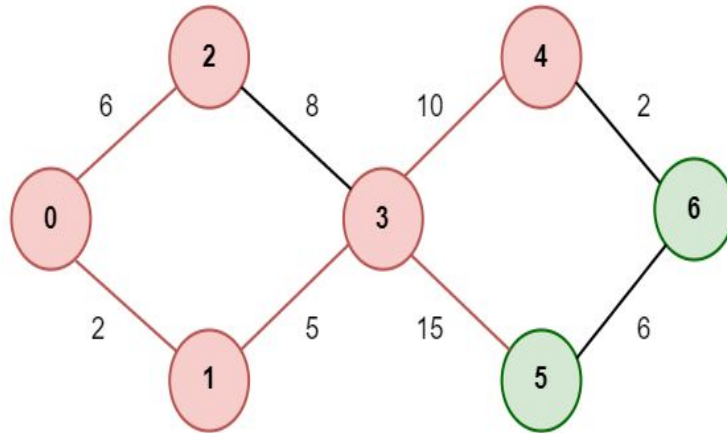
6: ∞

Dijkstra's Algorithm

Example of Dijkstra Algorithm

STEP 4

Mark Node 4 as Visited after considering the Optimal path and add the Distance



Unvisited Nodes

{0,1,2,3,4,5,6}

Distance:

0: 0 ✓

1: 2 ✓

2: 6 ✓

3: 7 ✓

4: 17 ✓

5: ∞

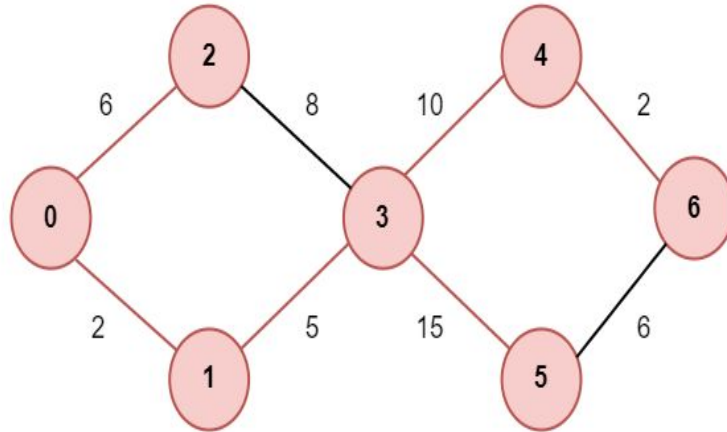
6: ∞

Dijkstra's Algorithm

Example of Dijkstra Algorithm

STEP 5

Mark Node 6 as Visited and add the Distance



Unvisited Nodes

{0,1,2,3,4,5,6}

Distance:

0: 0 ✓

1: 2 ✓

2: 6 ✓

3: 7 ✓

4: 17 ✓

5: 22 ✓

6: 19 ✓

Dijkstra's Algorithm

Complexity Analysis of Dijkstra Algorithm

Time complexity: $O(E \log V)$, where V is the number of vertices and E is the number of edges.

Auxiliary Space: $O(V)$, where V is the number of vertices and E is the number of edges in the graph.

Dijkstra's Algorithm Applications

- 1.To find the shortest path
- 2.In social networking applications
- 3.To find the locations in the map

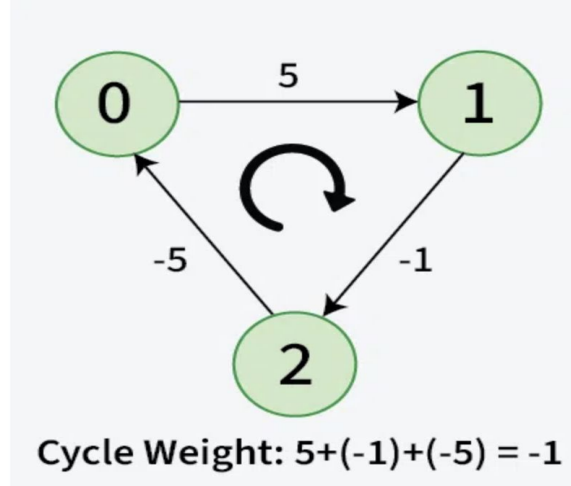
Bellman Ford Algorithm

The Bellman Ford Algorithm is a dynamic programming algorithm used to solve the Single Source Shortest Path problem. It is particularly useful for graphs with negative edge weights and can also detect negative weight cycles.

However, it is less efficient than Dijkstra's algorithm for graphs without negative weights.

How does the Bellman-Ford algorithm handle negative edge weights?

The Bellman-Ford algorithm is specifically designed to handle negative edge weights by iteratively relaxing all edges $V-1$ times (where V is the number of vertices). This iterative process ensures that even with negative weights, the shortest path distances are calculated correctly. Additionally, it has a mechanism to detect negative weight cycles, which are situations where a cycle's total weight is negative, making the shortest path undefined.



Bellman Ford Algorithm

Bellman Ford Algorithm steps:

1. Mark the source node with a current distance of 0 ($d[s]=0$) and assign infinity (∞) to all other nodes.
2. Set the unvisited node with the smallest current distance as the current node (u).
3. Repeat $V-1$ times (where V is the number of vertices):

For each neighbor v of u :

If ($d[u]+c[u,v]<d[v]$)
 $d[v]=d[u]+c[u,v]$

Here:

$c[u,v]$: Cost (weight) of the edge from u to v .

$d[v]$: Distance to v .

$d[u]$: Distance to u

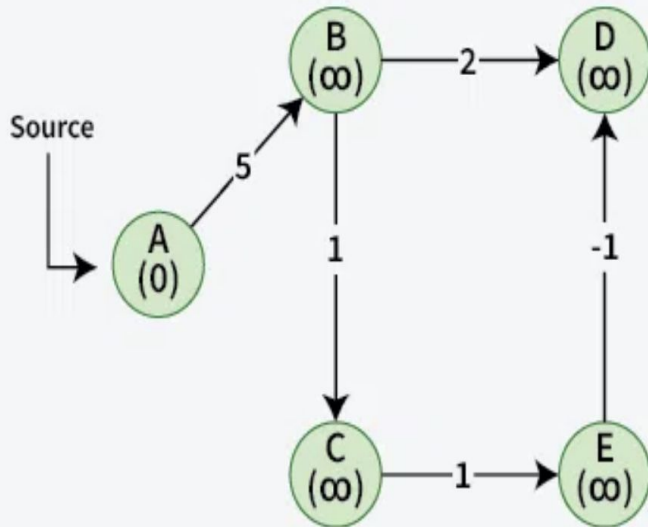
4. Negative Cycle Detection: After $V-1$ iterations, perform one more iteration over all edges. If you can still relax any edge (u,v) , it means a negative weight cycle exists, and the algorithm should terminate with an error or warning.

5. If no negative weight cycle is detected, return the shortest distances to all nodes.

Example of Bellman Ford Algorithm

01
Step

Initialize The Distance Array



u	v	wt
B	D	2
E	D	-1
C	E	1
B	C	1
A	B	5

Edges

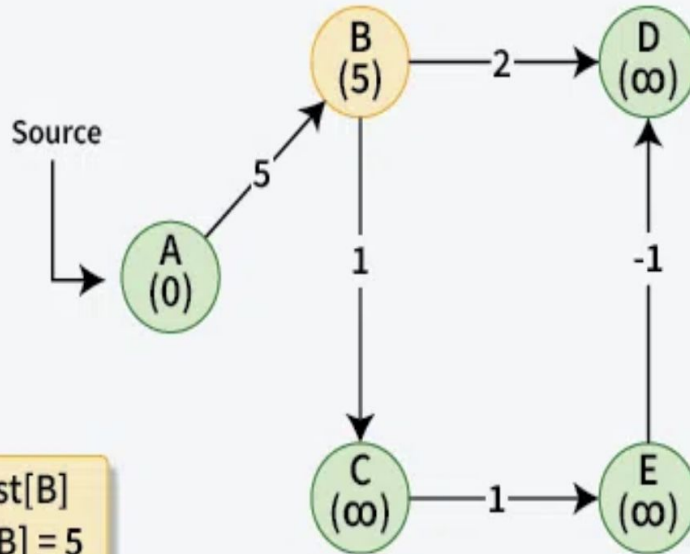
A	B	C	D	E
0	∞	∞	∞	∞

Distance Array

Example of Bellman Ford Algorithm

03
Step

1st Relaxation Of Edges



$\text{Dist}[A] + 5 < \text{Dist}[B]$
 $0 + 5 < (\infty), \text{Dist}[B] = 5$

Distance Array

A	B	C	D	E
0	∞	∞	∞	∞

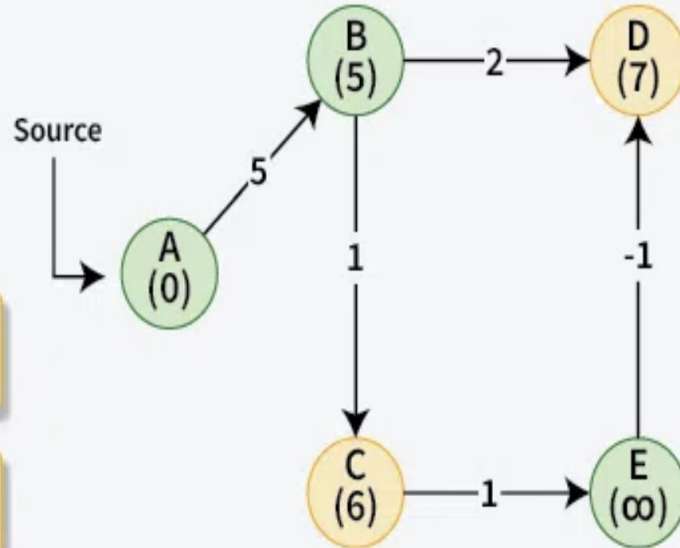
↓

A	B	C	D	E
0	5	∞	∞	∞

Example of Bellman Ford Algorithm

04
Step

2nd Relaxation Of Edges



$\text{Dist}[B] + 2 < \text{Dist}[D]$
 $5 + 2 < (\infty)$, $\text{Dist}[D] = 7$

$\text{Dist}[B] + 1 < \text{Dist}[C]$
 $5 + 1 < (\infty)$, $\text{Dist}[C] = 6$

Distance Array

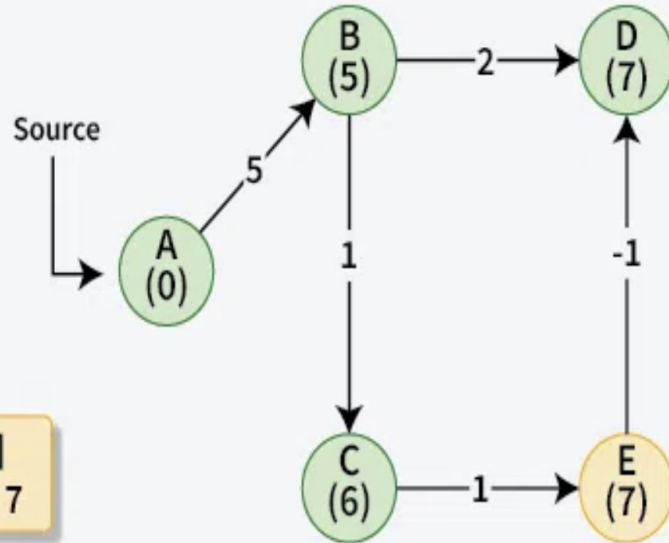
A	B	C	D	E
0	5	∞	∞	∞

A	B	C	D	E
0	5	6	7	∞

Example of Bellman Ford Algorithm

05
Step

3rd Relaxation Of Edges



$\text{Dist}[C] + 1 < \text{Dist}[E]$
 $6 + 1 < (\infty) \text{Dist}[E] = 7$

Distance Array

A	B	C	D	E
0	5	6	7	∞

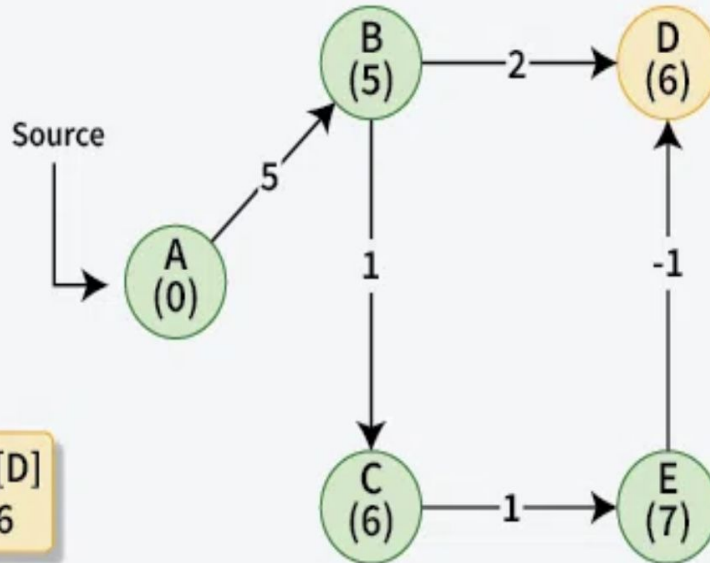


A	B	C	D	E
0	5	6	7	7

Example of Bellman Ford Algorithm

06
Step

4th Relaxation Of Edges



$\text{Dist}[E] + (-1) < \text{Dist}[D]$
 $7 + (-1) < 7, \text{Dist}[D] = 6$

Distance Array

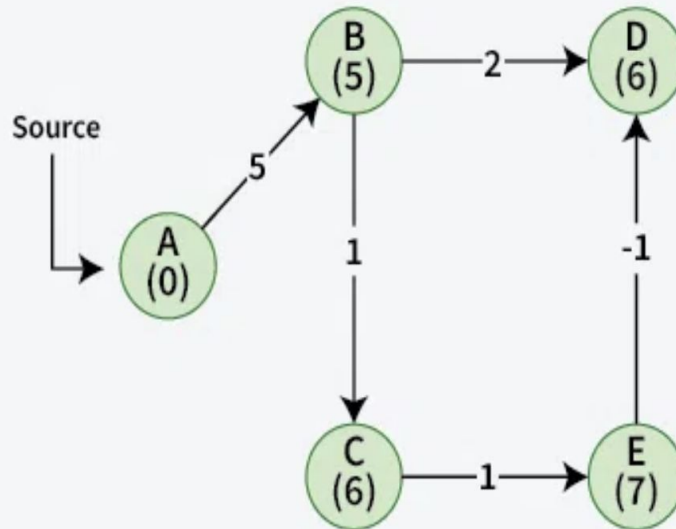
A	B	C	D	E
0	5	6	7	7

A	B	C	D	E
0	5	6	6	7

Example of Bellman Ford Algorithm

07
Step

5th relaxation to **detect** a negative weight cycle
Since no shorter distance for any node is found, the graph does not contain a negative weight cycle.



A	B	C	D	E
0	5	6	6	7

Distance Array

Time and Space Complexity of Bellman-Ford Algorithm

Time Complexity: $O(V \cdot E)$

V: Number of vertices

E: Number of edges

Space Complexity: $O(V+E)$

Applications of the Bellman-Ford Algorithm

Following are the applications of the bellman ford algorithm:

- 1.Examining a graph for the presence of negative weight cycles.
- 2.Using negative weights, find the shortest path in a graph.
- 3.Routing is a concept used in data networks.